DAPP 개발 실습 1

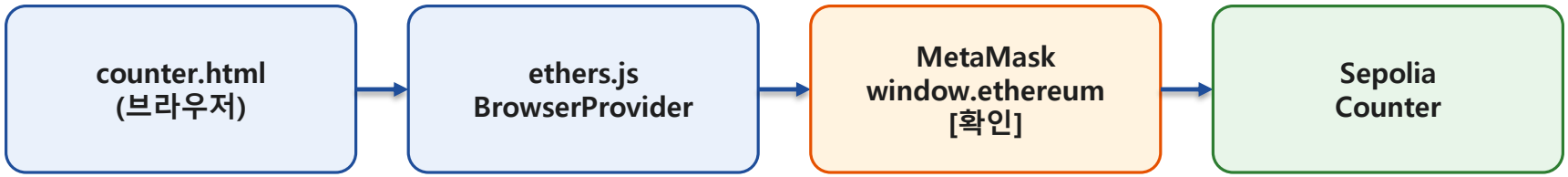
인하공업전문대학 컴퓨터정보공학과
최효현 교수

주요사항

- 1. 웹 페이지에서 Counter 호출
  - ✓ HTML + ethers.js + MetaMask
  - ✓ 웹 서버, counter.html 읽고 실행
  - ✓ 읽기(call), 쓰기(트랜잭션), 이벤트
  - ✓ 연습: add(n) 버튼
- 준비
  - ✓ hardhat-test 프로젝트 (Sepolia에 Counter 배포 완료)
  - ✓ MetaMask (Sepolia, 실습 계정)
- 2. 초급 예제: 홀짝 게임
  - ✓ 새 프로젝트 hardhat-test2 만들기
  - ✓ OddEven.sol, 배포 (준비금)
  - ✓ odd-even.html: ETH 보내기, 결과 이벤트, 준비금 회수
  - ✓ 가스 예상이 모자라는 문제
- 3. 변화 예제
  - ✓ ① 3배 ② 내 전적 ③ 주사위 ④ 베팅 금액
  - ✓ 생각해 보기: 블록체인의 랜덤
- 부록: 전체 소스

웹 페이지

❑ 지금까지는 터미널, 이제는 웹 페이지에서 Counter 호출



	interact.js (로컬)	interact-wallet.js (Sepolia)	counter.html (이번 실습)
실행	node scripts/interact.js	node scripts/interact-wallet.js	브라우저에서 페이지 열기
Provider	JsonRpcProvider(127.0.0.1)	JsonRpcProvider(Sepolia RPC)	BrowserProvider(window.ethereum)
Signer (서명)	Hardhat Node 계정	ethers.Wallet (환경 변수 개인 키)	MetaMask (확인 창에서 승인)
개인 키	필요 없음	코드 밖 환경 변수	MetaMask 안에만 있음

- ✓ **dApp(웹 서비스)의 기본 구조: 웹 페이지는 개인 키를 갖지 않고, 서명은 사용자의 지갑이 함**
- ✓ **ABI, Contract, count(), increment()는 interact.js와 같음**

웹 페이지

❑ 파일 구성과 웹 서버

파일	역할	학생
web/counter.html	Counter 화면 + ethers.js 코드, Starter에 포함	제공 (읽고 실행)
scripts/serve.js	작은 웹 서버 (http://localhost:3000), Starter에 포함	수정하지 않음
ignition/deployments/chain-…/deployed_addresses.json	배포 주소 (Ignition이 만든 파일)	그대로 사용

❑ 왜 웹 서버가 필요한가

- ✓ **더블클릭(file:///)으로 열면 브라우저가 막음 → MetaMask 연결, 다른 파일 읽어 오기**
- ✓ **페이지는 열린 뒤 ethers.js(import, 도구)와 배포 기록(fetch, 주소)을 다시 요청함**
- ✓ **serve.js는 web/, ethers.js, 배포 기록, artifacts 폴더만 보내 줌 (다른 파일은 404)**

```
PS C:\blockchain\hardhat-test> node scripts/serve.js
웹 페이지: http://localhost:3000   (끝내려면 Ctrl + C)
```

웹 페이지

❑ web/counter.html 읽기 ① 화면 (HTML)

```
<body>
  <script>
    // 더블클릭(file://)으로 열면 MetaMask, import 가 동작하지 않음 → 웹 서버 주소로 열도록 안내
    if (location.protocol === "file:") {
      document.write('<p style="color:#c00000; font-weight:bold">더블클릭하지 말고, node scripts/serve.js 를 실행한 뒤 http://localhost:3000/web/' +
        location.pathname.split("/").pop() + ' 로 여세요</p>');
    }
  </script>
  <h1>Counter (HTML + ethers.js + MetaMask)</h1>

  <button id="connectBtn">MetaMask 연결</button>
  <p>계정: <span id="account">-</span></p>
  <p>네트워크 chain ID: <span id="network">-</span></p>
  <p>Contract 주소: <span id="address">-</span></p>

  <h2>count = <span id="count">-</span></h2>
  <button id="incBtn" disabled>increment()</button>
  <p id="status"></p>

  <h3>Increment 이벤트</h3>
  <ul id="events"></ul>

  <script type="module">
    …
```

- 첫 <script> (module 아님)
  - ✓ 더블클릭(file://)으로 열면 빨간 안내 문구
- id 로 찾아서 바꿈
  - ✓ `connectBtn` 연결 버튼
  - ✓ `account`, `network`, `address`
  - ✓ `count` 숫자, `incBtn` 버튼
  - ✓ `status` 진행 상황, 오류
  - ✓ `events` 이벤트 목록
- script type="module"
  - ✓ import 문법 사용

웹 페이지

❑ web/counter.html 읽기 ② ethers.js, ABI, MetaMask 감시

```
// 1. ethers.js 불러오기 (node_modules 안의 브라우저용 파일)
import { ethers } from "/node_modules/ethers/dist/ethers.min.js";

// 2. ABI: interact.js 와 같음 (+ 이벤트)
const abi = [
  "function count() view returns (uint256)",
  "function increment()",
  "function add(uint256)",
  "event Increment(uint256 newValue)"
];

let counter;
const $ = (id) => document.getElementById(id);

// MetaMask 에서 네트워크를 바꾸면 새로 고침, 연결된 뒤 계정을 바꿔도 새로 고침 (페이지를 열 때 바로 등록)
// (처음 연결을 승인할 때도 accountsChanged 가 오므로, 연결이 끝난 뒤에만 새로 고침)
window.ethereum?.on("chainChanged", () => location.reload());
window.ethereum?.on("accountsChanged", () => { if (counter) location.reload(); });
```

- ✓ **ABI는 interact.js 와 같은 사람이 읽을 수 있는 형태 + 이벤트**
- ✓ **chainChanged: MetaMask에서 네트워크를 바꾸면 새로 고침 → 새 네트워크로 다시 연결**
- ✓ **accountsChanged: 연결된 뒤 계정을 바꾸면 새로 고침**
  - ➢ **처음 연결을 승인할 때도 이 이벤트가 오므로, 연결이 끝난 뒤(counter 가 있을 때)만**
- ✓ **페이지를 열 때 바로 등록 → 연결에 실패한 뒤 네트워크를 바꿔도 새로 고침됨**

웹 페이지

❑ web/counter.html 읽기 ③ Provider, Signer, 배포 주소

```
$("connectBtn").onclick = async () => {
  try {
    // 3. Provider: MetaMask (window.ethereum)
    if (!window.ethereum) throw new Error("MetaMask가 설치되어 있지 않습니다");
    const provider = new ethers.BrowserProvider(window.ethereum);

    // 4. Signer: MetaMask 계정 (처음 한 번 연결 승인 창)
    await provider.send("eth_requestAccounts", []);
    const signer = await provider.getSigner();
    const { chainId } = await provider.getNetwork();

    // 5. 배포 주소: Ignition 배포 기록 파일을 웹 서버에서 읽기
    const res = await fetch(`/ignition/deployments/chain-${chainId}/deployed_addresses.json`);
    if (!res.ok) {
      // 이 네트워크에는 배포 기록이 없음 → MetaMask 에 Sepolia(0xaa36a7 = 11155111)로 바꿔 달라고 요청
      $("status").textContent = `chain ${chainId} 에 배포 기록이 없습니다 → MetaMask에서 Sepolia로 전환해 주세요`;
      await provider.send("wallet_switchEthereumChain", [{ chainId: "0xaa36a7" }]);
      return;   // 전환되면 chainChanged → 새로 고침 → 연결 버튼 다시 누르기
    }
    const address = (await res.json())["CounterModule#Counter"];
        …
```

- ✓ `BrowserProvider(window.ethereum)`: MetaMask를 통해 연결 (RPC URL을 코드에 안 씀)
- ✓ `eth_requestAccounts`: 연결 승인 창 (이미 연결된 사이트면 창 없이 바로 계정)
- ✓ chain ID로 배포 기록 폴더를 고름 → 기록이 없으면 `wallet_switchEthereumChain` 으로 Sepolia 전환 요청
  - ➢ MetaMask에 전환 확인 창 → 전환되면 chainChanged → 새로 고침 → 연결 버튼 다시 누르기

웹 페이지

❑ web/counter.html 읽기 ④ Contract, 이벤트

```
    // 6. Contract: interact.js 와 같음
    counter = new ethers.Contract(address, abi, signer);

    $("account").textContent = await signer.getAddress();
    $("network").textContent = chainId;
    $("address").textContent = address;
    $("connectBtn").disabled = true;
    $("incBtn").disabled = false;
    $("status").textContent = "";   // 이전 오류 문구 지우기
    await showCount();

    // 7. 이벤트: 누가 increment() 를 호출하든 화면에 표시
    counter.on("Increment", (newValue) => {
      $("events").innerHTML += `<li>Increment(${newValue})</li>`;
      $("count").textContent = newValue;
    });
  } catch (e) {
    $("status").textContent = e.shortMessage ?? e.message;
  }
};
```

- ✓ Contract 만들기는 interact.js 와 같음 (주소 + ABI + Signer)
- ✓ 연결에 성공하면 이전 오류 문구를 지움

웹 페이지

❑ web/counter.html 읽기 ⑤ 읽기(call)와 쓰기(트랜잭션)

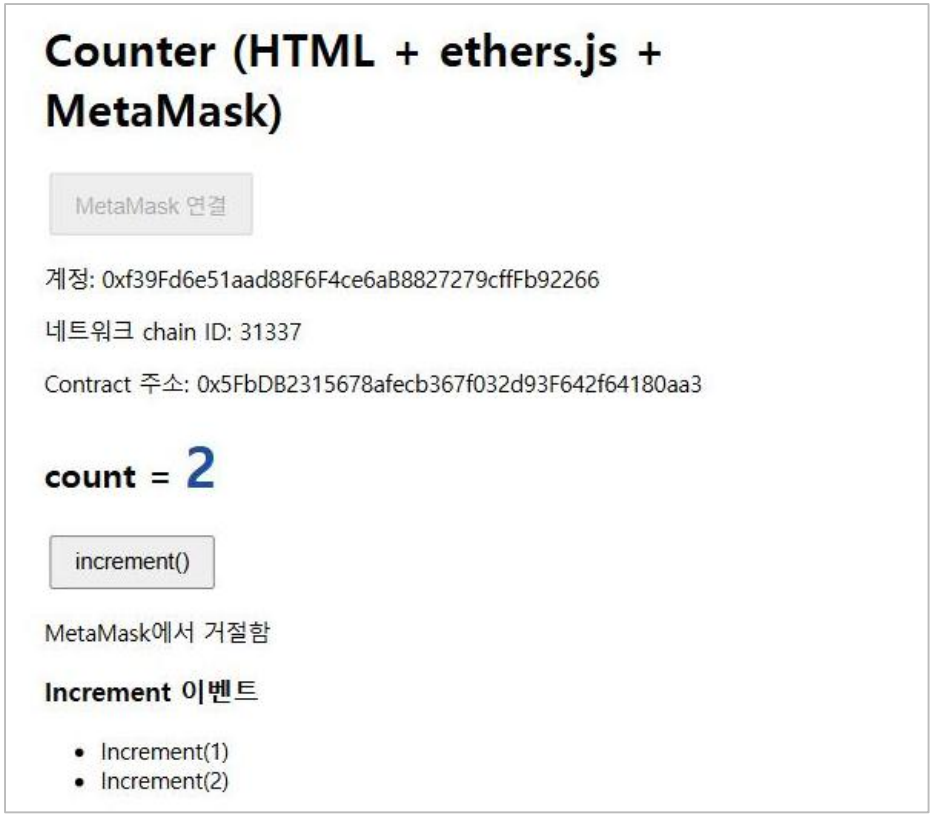
```
// view 함수 → call (트랜잭션 X, MetaMask 창 X, 가스비 X)
async function showCount() {
  $("count").textContent = await counter.count();
}

// 상태 변경 함수 → 트랜잭션 (MetaMask 확인 창, 가스비 O)
$("incBtn").onclick = async () => {
  try {
    $("status").textContent = "MetaMask에서 확인해 주세요";
    const tx = await counter.increment();
    $("status").textContent = "블록에 담기기를 기다리는 중… " + tx.hash;
    const receipt = await tx.wait();
    $("status").textContent = `완료: 블록 ${receipt.blockNumber}, 가스 ${receipt.gasUsed}`;
    await showCount();
  } catch (e) {
    $("status").textContent = e.code === "ACTION_REJECTED" ? "MetaMask에서 거절함" : (e.shortMessage ?? e.message);
  }
};
```

	counter.count()	counter.increment()
MetaMask 창	없음	확인 창 (가스비 표시)
결과	값을 바로 받음	tx → tx.wait() → receipt
거절하면	-	e.code === "ACTION_REJECTED"

웹 페이지

❑ 실행 화면



❑ 순서

- ✓ MetaMask 연결 → 승인
- ✓ count 표시 (call)
- ✓ increment() → MetaMask 확인
- ✓ 완료: 블록 번호, 가스
- ✓ Increment 이벤트 표시

❑ 확인해 보기

- ✓ node scripts/interact-wallet.js 로 increment → 페이지가 이벤트로 바뀜
- ✓ MetaMask에서 거절 → "MetaMask에서 거절함"

※ 로컬 테스트 노드로 확인한 화면 예시 (chain ID 31337). Sepolia에서는 11155111, 내 계정과 Contract 주소

웹 페이지

❑ 자주 나는 문제

화면에 보이는 것	원인	해결
MetaMask가 설치되어 있지 않습니다	MetaMask 없는 브라우저	MetaMask 가 있는 브라우저에서 열기
사이트에 연결할 수 없음	serve.js 를 켜지 않음	node scripts/serve.js (터미널 하나 더)
버튼을 눌러도 반응 없음 / 빨간 안내 문구	더블클릭(file://)으로 실행시킴	주소창에 http://localhost:3000/web/counter.html
chain … 배포 기록이 없습니다 → Sepolia로 전환	MetaMask 네트워크가 Sepolia가 아님	MetaMask 전환 확인 창에서 승인 → 연결 다시
MetaMask에서 거절함	확인 창에서 거절	다시 누르고 확인
insufficient funds	Sepolia ETH 부족	Faucet에서 받기
could not decode result data	그 주소에 Counter가 없음	배포 기록, 네트워크 확인

웹 페이지

- **연습: add(n) 버튼 추가하기**
  - ✓ 숫자 입력 칸과 add(n) 버튼을 만들고, 누르면 그 숫자만큼 증가
  - ✓ ABI에는 이미 `function add(uint256)` 이 있음
- **힌트**
  - ✓ HTML: `<input id="addInput" type="number" value="5">` 와 `<button id="addBtn" disabled>`
  - ✓ 연결이 끝나면 버튼 켜기: `$("addBtn").disabled = false`
  - ✓ 입력 값은 문자열 → `BigInt($("addInput").value)` 로 바꿔서 전달
  - ✓ increment() 버튼 코드를 복사해서 `counter.add(…)` 로 바꾸기
- **확인: 5를 넣고 누르면 count가 5 증가, 이벤트 Increment(…) 표시**

❑ 연습 정답 (answers/counter-add.html)

```
<input id="addInput" type="number" value="5" min="1" style="width:60px; font-size:16px">
<button id="addBtn" disabled>add(n)</button>
```

```
// 연습 정답: add(n) – 입력한 숫자는 문자열 → BigInt 로 바꿔서 전달
$("addBtn").onclick = async () => {
  try {
    $("status").textContent = "MetaMask에서 확인해 주세요";
    const tx = await counter.add(BigInt($("addInput").value));
    $("status").textContent = "블록에 담기기를 기다리는 중… " + tx.hash;
    await tx.wait();
    $("status").textContent = "완료";
    await showCount();
  } catch (e) {
    $("status").textContent = e.code === "ACTION_REJECTED" ? "MetaMask에서 거절함" : (e.shortMessage ?? e.message);
  }
};
```

※ 연결 코드에 $("addBtn").disabled = false; 한 줄 추가. Counter.sol의 add()도 Increment 이벤트를 내므로 이벤트 목록에 표시됨

```
$("incBtn").disabled = false;
$("status").textContent = "";
$("addBtn").disabled = false;      //← 여기에 추가
```

새 프로젝트

- 홀짝 게임부터는 새 프로젝트 hardhat-test2 에서
  - ✓ 프로젝트 하나 = 앱(서비스) 하나: Counter와 Storage처럼 한 앱의 컨트랙트는 한 프로젝트, 다른 앱은 다른 프로젝트
  - ✓ 학생 프로젝트도 자기 폴더를 새로 만들어 시작 → 미리 연습

```
PS C:\> cd C:\blockchain
PS C:\blockchain> git clone https://github.com/Inhatc-BlockChain/hardhat-test2.git
PS C:\blockchain> cd hardhat-test2
PS C:\blockchain\hardhat-test2> npm ci
```

  - ✓ 두 번째 Starter 저장소 **hardhat-test2**: 홀짝, 투표 컨트랙트와 웹 페이지, serve.js 가 들어 있음 (ZIP이면 폴더 이름 hardhat-test2-main → hardhat-test2)
  - ✓ npm ci 를 다시 함 (node_modules 는 프로젝트마다 따로, 약 200MB)
  - ✓ VS Code → File → Open Folder → hardhat-test2 → **새 창**으로 열기
  - ✓ 명령을 치기 전에 터미널 경로 확인: `PS C:\blockchain\hardhat-test2>`
    - ➢ hardhat-test 에서 배포하면 기록이 hardhat-test 에 남아 웹 페이지가 "배포 기록이 없습니다" 로 나옴

새 프로젝트

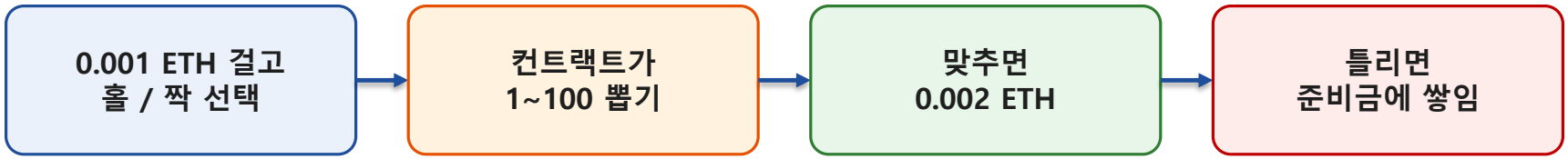
❑ 프로젝트마다 따로인 것, 같이 쓰는 것

	프로젝트마다 따로	모든 프로젝트가 같이 씀
node_modules, hardhat.config.ts	O	
contracts, artifacts	O	
배포 기록 ignition/deployments/	O (hardhat-test2 에는 Counter 기록 없음)	
keystore (RPC URL, 개인 키)		O (Windows 사용자별 → 다시 set 안 함)
MetaMask 계정, nonce		O
Sepolia에 배포된 컨트랙트		O (블록체인에 그대로)

- ✓ **counter.html 은 hardhat-test 에서 (Counter 배포 기록이 그곳에 있음)**
- ✓ **웹 서버도 프로젝트마다: hardhat-test2 폴더에서 `node scripts/serve.js` (실행한 폴더의 파일을 보냄)**

홀짝 게임

❑ 초급 예제: 홀짝 게임 규칙 (hardhat-test2)



파일	내용
contracts/OddEven.sol	[제공] 게임 규칙: ETH 받기(payable), 숫자 뽑기, 상금 보내기
ignition/modules/OddEven.ts	[제공] 배포 + 상금 준비금 0.005 ETH 함께 보내기
web/odd-even.html	[제공] counter.html과 같은 구조 + ETH 보내기, 결과 이벤트 읽기

❑ 새로 배우는 것

- ✓ payable 함수에 ETH 보내기: `{ value }`, 컨트랙트가 ETH 보내기: `call{value: …}`
- ✓ 트랜잭션 결과를 이벤트로 받기: `receipt.logs` → Played(뽑힌 숫자, 승패)
- ✓ 블록체인에서 "랜덤"이 어려운 이유

홀짝 게임

❑ contracts/OddEven.sol ① 게임 규칙

```
contract OddEven {
    address public owner;
    uint256 public constant BET = 0.001 ether;

    event Played(address indexed player, bool pickOdd, uint256 number, bool win);

    constructor() payable {
        owner = msg.sender;
    }

    function play(bool pickOdd) external payable {
        require(msg.value == BET, "send 0.001 ETH");
        require(address(this).balance >= 2 * BET, "not enough prize money");

        // 블록 시간으로 만든 "가짜 랜덤" → 누구나 아는 값이라 예측할 수 있음 (생각해 보기)
        uint256 number = uint256(
            keccak256(abi.encodePacked(block.timestamp, msg.sender))
        ) % 100 + 1;
        bool win = (number % 2 == 1) == pickOdd;
        emit Played(msg.sender, pickOdd, number, win);

        uint256 prize = 2 * BET;
        if (!win) prize = 1;   // 지면 1 wei: 이기든 지든 송금 → 가스량이 거의 같음 (가스 예상이 맞음)
        (bool ok, ) = payable(msg.sender).call{value: prize}("");
        require(ok, "transfer failed");
    }
    …
```

홀짝 게임

❑ contracts/OddEven.sol ② 코드 설명

코드	의미
`constructor() payable`	배포할 때 ETH(상금 준비금)를 함께 받음
`uint256 public constant BET`	고정 값 0.001 ether, public → 웹에서 game.BET() 로 읽기
`function play(bool) payable`	호출할 때 ETH를 함께 받는 함수, 받은 금액 = msg.value
`require(msg.value == BET, …)`	0.001 ETH가 아니면 revert (보낸 ETH도 돌려줌)
`address(this).balance`	컨트랙트가 가진 ETH (준비금 + 받은 판돈)
`keccak256(…) % 100 + 1`	블록 시간(block.timestamp)과 내 주소로 만든 1~100 숫자 (진짜 랜덤 아님 → 생각해 보기)
`emit Played(…)`	누가, 무엇을 골랐고, 무슨 숫자가 나왔는지 기록 (player는 indexed)
`payable(msg.sender).call{value: prize}("")`	ETH 보내기: 이기면 0.002 ETH, 지면 1 wei (이유는 뒤에서, transfer 대신 call 권장)
`withdraw(), receive()`	owner만 준비금 회수, 아무나 준비금 추가

홀짝 게임

❑ 배포: ignition/modules/OddEven.ts

```
import { buildModule }
  from "@nomicfoundation/hardhat-ignition/modules";

// 배포할 때 상금 준비금을 함께 보냄 (기본 0.005 ETH = 2배 상금 2.5번)
export default buildModule("OddEvenModule", (m) => {
  const reserve = m.getParameter("reserve", 5n * 10n ** 15n);
  const game = m.contract("OddEven", [], { value: reserve });
  return { game };
});
```

```
PS C:\blockchain\hardhat-test2> npx hardhat build
PS C:\blockchain\hardhat-test2> npx hardhat ignition deploy ignition/modules/OddEven.ts --network sepolia
[hardhat-keystore] Enter the password: ********
√ Confirm deploy to network sepolia (11155111)? ... yes
[ OddEvenModule ] successfully deployed 🚀
Deployed Addresses
OddEvenModule#OddEven - 0x…(각자 다른 주소)
```

- ✓ `{ value: reserve }`: 배포 트랜잭션에 준비금 0.005 ETH가 함께 감 → 내 계정에서 빠져나감
- ✓ keystore 는 그대로 사용 (비밀번호만 입력), 배포 기록은 hardhat-test2 에 따로 생김 (Counter 없음)
- ✓ 실습이 끝나면 남은 준비금은 웹 페이지의 [남은 준비금 회수] 버튼으로 (owner만)

홀짝 게임

web/odd-even.html ① counter.html과 다른 곳

```
const abi = [
  "function play(bool pickOdd) payable",
  "function BET() view returns (uint256)",
  "function withdraw()",
  "event Played(address indexed player, bool pickOdd, uint256 number, bool win)"
];
```

```
const address = (await res.json())["OddEvenModule#OddEven"];
if (!address) throw new Error("OddEven 이 배포되지 않았습니다");
game = new ethers.Contract(address, abi, signer);
```

	counter.html	odd-even.html
ABI	count, increment, add, Increment	play (payable), BET, withdraw, Played
배포 기록에서 찾는 이름	CounterModule#Counter	OddEvenModule#OddEven
버튼	increment()	홀, 짝 → play(true), play(false), 준비금 회수 → withdraw()
화면	count 값	뽑힌 숫자, 승패, 잔액, 최근 게임

※ Provider, Signer, chain ID 로 배포 기록 읽기는 counter.html 과 같음

홀짝 게임

❑ web/odd-even.html ② ETH 보내고 결과 받기

```
async function play(pickOdd) {
  try {
    $("result").textContent = "";
    $("status").textContent = "MetaMask에서 확인해 주세요";
    // payable 함수: { value } 로 ETH 를 함께 보냄
    const value = await game.BET();
    // 가스는 보내기 전에 미리 실행해서 정함 → 실패할 거래(금액 틀림, 상금 부족)는 이유와 함께 멈춤, 보내지 않음
    const tx = await game.play(pickOdd, { value });
    $("status").textContent = "블록에 담기기를 기다리는 중…";
    const receipt = await tx.wait();

    // receipt.logs 에서 Played 이벤트 찾기 → 뽑힌 숫자, 승패
    const ev = receipt.logs.map((log) => game.interface.parseLog(log)).find((p) => p?.name === "Played");
    const { number, win } = ev.args;
    $("result").textContent = `뽑힌 숫자: ${number} (${number % 2n === 1n ? "홀" : "짝"})`;
    $("status").textContent = win ? "🎉 이겼습니다! 0.002 ETH를 받았습니다" : "❌ 졌습니다";
  } catch (e) {
    $("status").textContent = e.code === "ACTION_REJECTED" ? "MetaMask에서 거절함" : (e.reason ?? e.shortMessage ?? e.message);
  }
  await refresh();
}
```

- ✓ `{ value }`: payable 함수에 ETH 보내기 (MetaMask 확인 창에 0.001 ETH + 가스비 표시)
- ✓ 가스는 ethers.js와 MetaMask가 미리 실행해서 정함 → 실패할 거래는 이유("not enough prize money" 등)와 함께 멈춤
- ✓ `receipt.logs` + `parseLog`: 블록에 담긴 뒤 정해진 결과(숫자, 승패)를 이벤트에서 읽기

홀짝 게임

- 가스 예상이 모자라는 문제: 왜 지면 1 wei를 보내나

이길 때만 상금을 보내던 컨트랙트 (Sepolia 실제 기록)	가스	결과
가스 한도 (MetaMask가 미리 실행해서 정함: 졌을 때)	24,453	
실제로 졌을 때 쓴 가스	24,114	성공
실제로 이겼을 때 필요한 가스 (상금 보내기 call 포함)	36,450	out of gas → 실패

- 원인: 숫자(승패)는 트랜잭션이 **블록에 담길 때** 정해짐
  - ✓ 미리 실행은 지금 블록의 시간으로, 실제 실행은 다음 블록(12초 뒤)의 시간으로 계산 → 승패가 바뀔 수 있음
  - ✓ 웹에서 가스 한도를 넉넉히 정해도 MetaMask가 자기 예상값으로 바꿀 때가 있음 (Sepolia에서 확인)
- 해결: 컨트랙트에서 이기든 지든 송금 (지면 1 wei) → 가스량이 거의 같음 (차이 8)
  - ✓ 확인: 테스트 노드에서 가스 한도 = 예상값 그대로 홀짝, 주사위 180번 실행 → 실패 0
- 규칙: 결과에 따라 **실행하는 코드가 크게 달라지면** 가스 예상이 빗나갈 수 있음

홀짝 게임

❑ 최근 게임 목록 (이벤트 읽기)과 실행 화면

```
// 최근 5000 블록의 Played 이벤트 (가스비 X)
const latest = await provider.getBlockNumber();
const logs = await game.queryFilter(game.filters.Played(),
Math.max(0, latest - 5000));
$("history").innerHTML = logs.reverse().slice(0, 10).map((l) =>
  `<li>${l.args.player.slice(0, 6)}… ${l.args.pickOdd ? "홀" : "짝"}
→ ${l.args.number} ${l.args.win ? "🎉" : "❌"}</li>`
).join("");
```

- ✓ `queryFilter`: 지난 이벤트 검색 (트랜잭션 X, 가스비 X)
- ✓ 최근 5000 블록만 (공개 RPC는 한 번에 검색할 수 있는 범위가 정해져 있음)
- ✓ 반 전체가 같은 게임 주소를 쓰면 다른 사람 결과도 보임
- ✓ 판돈은 준비금에 쌓이고, 이기면 준비금에서 나감



※ 로컬 테스트 노드 화면 예시

홀짝 게임

❑ web/odd-even.html ③ 남은 준비금 회수 (withdraw)

```
<p>배포한 계정만: <button id="withdrawBtn" disabled>남은 준비금 회수</button></p>
```

```
// 남은 준비금 회수: 배포한 계정(owner)만 가능 → 다른 계정이면 미리 실행에서 "only owner" 로 멈춤, 보내지 않음
$("withdrawBtn").onclick = async () => {
  try {
    $("status").textContent = "MetaMask에서 확인해 주세요";
    const tx = await game.withdraw();
    $("status").textContent = "블록에 담기기를 기다리는 중…";
    await tx.wait();
    $("status").textContent = "남은 준비금을 회수했습니다";
  } catch (e) {
    $("status").textContent = e.code === "ACTION_REJECTED" ? "MetaMask에서 거절함" : (e.reason ?? e.shortMessage ?? e.message);
  }
  await refresh();
};
```

- ✓ `withdraw()` 는 owner(배포한 계정)만: `require(msg.sender == owner, "only owner")`
- ✓ 다른 계정으로 누르면 미리 실행(가스 예상)에서 "only owner" 로 멈춤 → 트랜잭션을 보내지 않음, 가스비 X
- ✓ 컨트랙트를 고쳐 **다시 배포하기 전에** 회수: 다시 배포하면 페이지가 새 주소를 씀 → 예전 게임의 준비금은 이 버튼으로 못 꺼냄
- ✓ 변화 예제 페이지(내 전적, 베팅 금액, 주사위)에도 같은 버튼

변화 예제

❑ 홀짝 게임 바꿔 보기

	바꿀 것	바꿀 파일	다시 배포	배우는 것
① **3배**	맞추면 3배	새 파일 OddEven3x.sol	O (새 이름)	상금 계산, 준비금 확인
② **내 전적**	내 승, 패 표시	odd-even.html	X	indexed 이벤트 필터
③ **주사위**	1~6 맞추면 5배	새 컨트랙트 + 웹	O	uint8, % 6, 버튼 만들기
④ **베팅 금액**	0.001~0.005 자유	새 파일 OddEvenBet.sol + 웹	O (새 이름)	msg.value 범위, parseEther

❑ 컨트랙트를 고치면 새로 배포해야 함 (불변성)

- ✓ 같은 모듈로 다시 배포 → Nothing new to deploy
- ✓ `--reset` 은 hardhat-test2 의 이 네트워크 기록을 모두 지움 → 원래 홀짝, 주사위 기록도 사라짐
- ✓ → **모듈 이름을 바꿔서** 배포: 기존 기록은 두고 새 기록만 추가됨 (실제 실행으로 확인)
- ✓ 새 파일로 만들 때는 **컨트랙트 이름도 바꿈**: 같은 이름이 둘이면 배포 오류 (HHE1001)

변화 예제

❑ ① 맞추면 3배: 새 파일 OddEven3x.sol (OddEven.sol 복사)

```
// contracts/OddEven3x.sol : 이름을 바꿈 (OddEven 은 이미 있음, 같은 이름이 둘이면 배포 오류)
contract OddEven3x {
…
    require(address(this).balance >= 3 * BET, "not enough prize money");   // 2 → 3
…
    uint256 prize = 3 * BET;                                               // 2 → 3
```

```
// ignition/modules/OddEven3x.ts : 모듈 이름, 컨트랙트 이름을 바꿈
export default buildModule("OddEven3xModule", (m) => {
  …
  const game = m.contract("OddEven3x", [], { value: reserve });
  …
```

```
// odd-even.html : 배포 기록에서 찾는 이름 (모듈#컨트랙트), 화면 문구
const address = (await res.json())["OddEven3xModule#OddEven3x"];
```

```
PS C:\blockchain\hardhat-test2> npx hardhat ignition deploy ignition/modules/OddEven3x.ts --network sepolia
…
OddEvenModule#OddEven - 0x…          ← 원래 게임 (그대로)
OddEven3xModule#OddEven3x - 0x…      ← 새 게임
```

※ 이길 확률은 그대로 1/2인데 3배 → 준비금이 빨리 줄어듦. 확률과 배당을 함께 생각하기

변화 예제

❑ ② 내 전적 표시 (answers/odd-even-record.html, 웹만 수정)

```
<p>내 전적: <b id="record">-</b></p>
```

```
  // 변화 예제 ②: 내 게임만 골라 보기 (player 가 indexed 라서 필터로 고를 수 있음)
  const mine = await game.queryFilter(game.filters.Played(signer.address), Math.max(0, latest - 5000));
  const wins = mine.filter((l) => l.args.win).length;
  $("record").textContent = `${wins}승 ${mine.length - wins}패`;
}
```

- ✓ `event Played(address indexed player, …)`: indexed 로 선언한 값은 **검색 조건**으로 쓸 수 있음
- ✓ `game.filters.Played(signer.address)` → 내가 한 게임만
- ✓ 컨트랙트를 바꾸지 않으므로 다시 배포할 필요 없음 (이벤트는 이미 블록에 기록되어 있음)
- ✓ 확인: 다른 계정으로 한 게임은 내 전적에 들어가지 않음

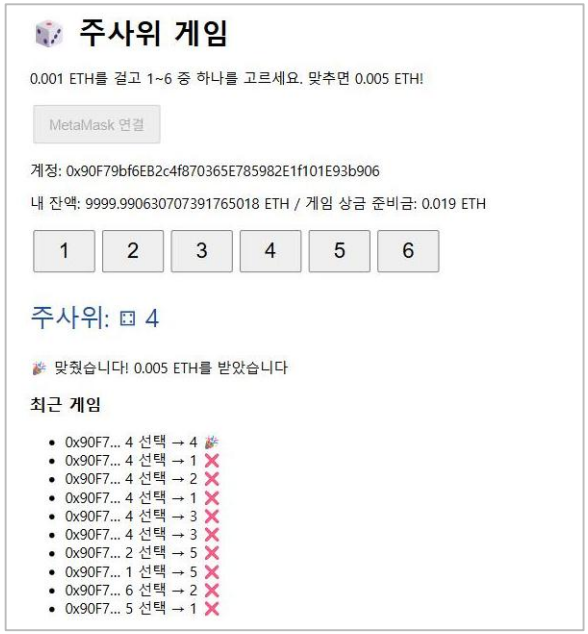
변화 예제

❑ ③ 주사위 게임: 1~6 맞추면 5배 (DiceGame.sol, dice.html)

```
function play(uint8 pick) external payable {
    require(pick >= 1 && pick <= 6, "pick 1~6");
    require(msg.value == BET, "send 0.001 ETH");
    require(address(this).balance >= PRIZE, "not enough prize money");

    uint8 number = uint8(
        uint256(keccak256(abi.encodePacked(block.timestamp, msg.sender))) % 6 + 1
    );
    bool win = (number == pick);
    emit Rolled(msg.sender, pick, number, win);

    uint256 prize = PRIZE;
    if (!win) prize = 1;   // 지면 1 wei: 이기든 지든 송금 → 가스량이 거의 같음 (가스
예상이 맞음)
    (bool ok, ) = payable(msg.sender).call{value: prize}("");
    require(ok, "transfer failed");
}
```



- ✓ `uint8 pick`, `require(pick >= 1 && pick <= 6)`
- ✓ `% 6 + 1` → 1~6, 맞출 확률 1/6 → 상금 5배 (PRIZE)
- ✓ 웹: 버튼 6개를 반복문으로 만들기, Rolled 이벤트, `DiceGameModule#DiceGame`

※ 배포: npx hardhat ignition deploy ignition/modules/DiceGame.ts --network sepolia (준비금 0.01 ETH)

변화 예제

❑ ④ 베팅 금액 자유: 0.001 ~ 0.005 ETH (OddEvenBet.sol)

```
contract OddEvenBet {   // ①처럼 새 이름
…
    require(msg.value >= BET && msg.value <= 5 * BET, "send 0.001 ~ 0.005 ETH");
    require(address(this).balance >= 2 * msg.value, "not enough prize money");
…
    uint256 prize = 2 * msg.value;
```

```
<!-- answers/odd-even-bet.html -->
<p>걸 금액: <input id="amount" value="0.002" style="width:80px; font-size:16px"> ETH</p>
```

```
const value = ethers.parseEther($("amount").value);   // "0.002" → wei (BigInt)
const tx = await game.play(pickOdd, { value });
```

- ✓ `address(this).balance` 에는 방금 받은 판돈도 들어 있음 → 2배를 보내도 되는지 확인
- ✓ 범위를 벗어나거나 준비금이 모자라면 미리 실행(가스 예상)에서 멈춤 → 보내지 않음, 가스비 X
- ✓ 모듈은 OddEvenBetModule (answers/OddEvenBet.ts) → ①과 함께 배포해도 됨
- ✓ 웹은 `"OddEvenBetModule#OddEvenBet"` 으로 주소를 찾음

※ 테스트 노드 확인: 0.0005, 0.006 → "send 0.001 ~ 0.005 ETH", 준비금 부족 → "not enough prize money" (보낸 트랜잭션 없음)

변화 예제

- 생각해 보기: 블록체인의 "랜덤"은 정말 랜덤일까?
  - ✓ `block.timestamp`(블록 시간, 12초 간격), `msg.sender` 는 모두 공개된 값 → 숫자를 미리 계산할 수 있음
    - ✓ 이더리움은 사실상 random 발생을 하기 어려움.
    - ✓ 모든 노드가 같은 결과를 내야 함
    - ✓ 수천 개의 노드가 같은 트랜잭션을 각자 실행해서 결과가 같아야 블록이 인정. 노드마다 결과가 달라지면 합의가 깨지기 때문
    - ✓ C의 랜덤 함수처럼 랜덤 발생을 위해 현재 시계처럼 컴퓨터마다 다른 값을 쓸 수 없음
  - ✓ 다른 컨트랙트가 같은 계산을 먼저 해 보고 이길 때만 play() 를 호출할 수 있음
    - ➢ 같은 트랜잭션 안에서 실행되므로 같은 블록 값으로 계산됨 → 질 것 같으면 호출하지 않음
  - ✓ 검증자(블록을 만드는 쪽)는 블록 값에 영향을 줄 수 있음

- 실제 서비스에서 쓰는 방법의 예
  - ✓ Chainlink VRF: 외부에서 증명할 수 있는 랜덤 값을 받아 옴

변화 예제

❑ 생각해 보기: OddEven 을 매번 이기는 컨트랙트 (Attacker)

```
contract Attacker {
    OddEven game;
    constructor(OddEven g) { game = g; }

    function attack() external payable {
        // OddEven 과 같은 계산: 같은 트랜잭션 → 같은 block.timestamp
        // OddEven 안에서 msg.sender = 이 컨트랙트 주소 = address(this)
        uint256 number = uint256(
            keccak256(abi.encodePacked(block.timestamp, address(this)))
        ) % 100 + 1;
        bool pickOdd = (number % 2 == 1);       // 이기는 쪽을 고름
        game.play{value: msg.value}(pickOdd);   // 바로 호출 → 항상 이김
                                                // → 이렇게 동시에 같은 블록에 들어갈 수 있게만 할 수 있다면 같은 효과
    }

    receive() external payable {}               // 상금 받기
}
```

- ✓ **attack() 과 play() 는 같은 트랜잭션 → 같은 블록의 block.timestamp 를 읽음**
- ✓ **웹, 지갑의 미리 계산과 다름: 실행되는 순간 두 컨트랙트가 같은 값을 봄 → 어긋나지 않음**

※ 설명용 코드 (실습 파일 아님): 상금은 Attacker 에 쌓임, 꺼내는 함수는 생략

정리

- 웹 페이지 (dApp)
  - ✓ BrowserProvider(window.ethereum) → Signer = MetaMask → 개인 키는 코드에 없음
  - ✓ 주소는 Ignition 배포 기록, ABI는 사람이 읽을 수 있는 형태 → interact.js와 같은 방법
  - ✓ view 함수는 call, 상태 변경 함수는 트랜잭션 (MetaMask 확인 창)
  - ✓ 이벤트: counter.on (실시간), queryFilter (지난 기록), receipt.logs (내 트랜잭션 결과)

- ETH를 주고받는 컨트랙트
  - ✓ payable, msg.value, address(this).balance, call{value: …}

부록

- 부록: 전체 소스
  - ✓ web/counter.html
  - ✓ contracts/OddEven.sol
  - ✓ ignition/modules/OddEven.ts
  - ✓ web/odd-even.html
  - ✓ scripts/serve.js
  - ✓ 연습 정답: hardhat-test/answers/counter-add.html
  - ✓ 변화 예제 정답: hardhat-test2/answers/ (OddEven3x.sol, OddEven3x.ts, OddEvenBet.sol, OddEvenBet.ts, odd-even-record.html, odd-even-bet.html), contracts/DiceGame.sol, web/dice.html
  - ✓ 파일 위치: Hardhat 실습 Starter / 교수자용 정답 / hardhat-test (counter.html), hardhat-test2 (홀짝)

전체 소스

web/counter.html (1/5) 1~27행

```
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <title>Counter</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 600px; margin: 30px auto; }
    button { font-size: 16px; padding: 8px 16px; margin: 4px; }
    #count { font-size: 40px; color: #1f4e9a; }
  </style>
</head>
<body>
  <script>
    // 더블클릭(file://)으로 열면 MetaMask, import 가 동작하지 않음 → 웹 서버 주소로 열도록 안내
    if (location.protocol === "file:") {
      document.write('<p style="color:#c00000; font-weight:bold">더블클릭하지 말고, node scripts/serve.js 를 실행한 뒤
http://localhost:3000/web/' +
        location.pathname.split("/").pop() + ' 로 여세요</p>');
    }
  </script>
  <h1>Counter (HTML + ethers.js + MetaMask)</h1>

  <button id="connectBtn">MetaMask 연결</button>
  <p>계정: <span id="account">-</span></p>
  <p>네트워크 chain ID: <span id="network">-</span></p>
  <p>Contract 주소: <span id="address">-</span></p>

  <h2>count = <span id="count">-</span></h2>
```

전체 소스

web/counter.html (2/5) 28~54행

```
<button id="incBtn" disabled>increment()</button>
<p id="status"></p>

<h3>Increment 이벤트</h3>
<ul id="events"></ul>

<script type="module">
  // 1. ethers.js 불러오기 (node_modules 안의 브라우저용 파일)
  import { ethers } from "/node_modules/ethers/dist/ethers.min.js";

  // 2. ABI: interact.js 와 같음 (+ 이벤트)
  const abi = [
    "function count() view returns (uint256)",
    "function increment()",
    "function add(uint256)",
    "event Increment(uint256 newValue)"
  ];

  let counter;
  const $ = (id) => document.getElementById(id);

  // MetaMask 에서 네트워크를 바꾸면 새로 고침, 연결된 뒤 계정을 바꿔도 새로 고침 (페이지를 열 때 바로 등록)
  // (처음 연결을 승인할 때도 accountsChanged 가 오므로, 연결이 끝난 뒤에만 새로 고침)
  window.ethereum?.on("chainChanged", () => location.reload());
  window.ethereum?.on("accountsChanged", () => { if (counter) location.reload(); });

  $("connectBtn").onclick = async () => {
```

전체 소스

web/counter.html (3/5) 55~81행

```
try {
  // 3. Provider: MetaMask (window.ethereum)
  if (!window.ethereum) throw new Error("MetaMask가 설치되어 있지 않습니다");
  const provider = new ethers.BrowserProvider(window.ethereum);

  // 4. Signer: MetaMask 계정 (처음 한 번 연결 승인 창)
  await provider.send("eth_requestAccounts", []);
  const signer = await provider.getSigner();
  const { chainId } = await provider.getNetwork();

  // 5. 배포 주소: Ignition 배포 기록 파일을 웹 서버에서 읽기
  const res = await fetch(`/ignition/deployments/chain-${chainId}/deployed_addresses.json`);
  if (!res.ok) {
    // 이 네트워크에는 배포 기록이 없음 → MetaMask 에 Sepolia(0xaa36a7 = 11155111)로 바꿔 달라고 요청
    $("status").textContent = `chain ${chainId} 에 배포 기록이 없습니다 → MetaMask에서 Sepolia로 전환해 주세요`;
    await provider.send("wallet_switchEthereumChain", [{ chainId: "0xaa36a7" }]);
    return;   // 전환되면 chainChanged → 새로 고침 → 연결 버튼 다시 누르기
  }
  const address = (await res.json())["CounterModule#Counter"];

  // 6. Contract: interact.js 와 같음
  counter = new ethers.Contract(address, abi, signer);

  $("account").textContent = await signer.getAddress();
  $("network").textContent = chainId;
  $("address").textContent = address;
  $("connectBtn").disabled = true;
```

전체 소스

web/counter.html (4/5) 82~108행

```
    $("incBtn").disabled = false;
    $("status").textContent = "";   // 이전 오류 문구 지우기
    await showCount();

    // 7. 이벤트: 누가 increment() 를 호출하든 화면에 표시
    counter.on("Increment", (newValue) => {
      $("events").innerHTML += `<li>Increment(${newValue})</li>`;
      $("count").textContent = newValue;
    });
  } catch (e) {
    $("status").textContent = e.shortMessage ?? e.message;
  }
};

// view 함수 → call (트랜잭션 X, MetaMask 창 X, 가스비 X)
async function showCount() {
  $("count").textContent = await counter.count();
}

// 상태 변경 함수 → 트랜잭션 (MetaMask 확인 창, 가스비 O)
$("incBtn").onclick = async () => {
  try {
    $("status").textContent = "MetaMask에서 확인해 주세요";
    const tx = await counter.increment();
    $("status").textContent = "블록에 담기기를 기다리는 중… " + tx.hash;
    const receipt = await tx.wait();
    $("status").textContent = `완료: 블록 ${receipt.blockNumber}, 가스 ${receipt.gasUsed}`;
```

전체 소스

web/counter.html (5/5) 109~116행

```
        await showCount();
      } catch (e) {
        $("status").textContent = e.code === "ACTION_REJECTED" ? "MetaMask에서 거절함" : (e.shortMessage ?? e.message);
      }
    };
  </script>
</body>
</html>
```

전체 소스

contracts/OddEven.sol (1/2) 1~27행

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

// 홀짝 게임: 0.001 ETH를 걸고 홀/짝 선택 → 컨트랙트가 1~100 숫자를 뽑음 → 맞추면 0.002 ETH
// 만든 사람이 상금 준비금을 넣어 둠
contract OddEven {
    address public owner;
    uint256 public constant BET = 0.001 ether;

    event Played(address indexed player, bool pickOdd, uint256 number, bool win);

    constructor() payable {
        owner = msg.sender;
    }

    function play(bool pickOdd) external payable {
        require(msg.value == BET, "send 0.001 ETH");
        require(address(this).balance >= 2 * BET, "not enough prize money");

        // 블록 시간으로 만든 "가짜 랜덤" → 누구나 아는 값이라 예측할 수 있음 (생각해 보기)
        uint256 number = uint256(
            keccak256(abi.encodePacked(block.timestamp, msg.sender))
        ) % 100 + 1;
        bool win = (number % 2 == 1) == pickOdd;
        emit Played(msg.sender, pickOdd, number, win);

        uint256 prize = 2 * BET;
```

전체 소스

❑ contracts/OddEven.sol (2/2) 28~41행

```
        if (!win) prize = 1;   // 지면 1 wei: 이기든 지든 송금 → 가스량이 거의 같음 (가스 예상이 맞음)
        (bool ok, ) = payable(msg.sender).call{value: prize}("");
        require(ok, "transfer failed");
    }

    function withdraw() external {
        require(msg.sender == owner, "only owner");
        (bool ok, ) = payable(owner).call{value: address(this).balance}("");
        require(ok, "transfer failed");
    }

    // 준비금 추가
    receive() external payable {}
}
```

전체 소스

ignition/modules/OddEven.ts 1~9행

```
import { buildModule }
  from "@nomicfoundation/hardhat-ignition/modules";

// 배포할 때 상금 준비금을 함께 보냄 (기본 0.005 ETH = 2배 상금 2.5번)
export default buildModule("OddEvenModule", (m) => {
  const reserve = m.getParameter("reserve", 5n * 10n ** 15n);
  const game = m.contract("OddEven", [], { value: reserve });
  return { game };
});
```

전체 소스

web/odd-even.html (1/5) 1~27행

```
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <title>홀짝 게임</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 600px; margin: 30px auto; }
    button { font-size: 16px; padding: 8px 16px; margin: 4px; }
    .pick { font-size: 24px; width: 120px; }
    #result { font-size: 28px; color: #1f4e9a; }
  </style>
</head>
<body>
  <script>
    // 더블클릭(file://)으로 열면 MetaMask, import 가 동작하지 않음 → 웹 서버 주소로 열도록 안내
    if (location.protocol === "file:") {
      document.write('<p style="color:#c00000; font-weight:bold">더블클릭하지 말고, node scripts/serve.js 를 실행한 뒤
http://localhost:3000/web/' +
        location.pathname.split("/").pop() + ' 로 여세요</p>');
    }
  </script>
  <h1>🎲 홀짝 게임</h1>
  <p>0.001 ETH를 걸고 홀/짝을 고르세요. 컨트랙트가 1~100 숫자를 뽑고, 맞추면 0.002 ETH!</p>

  <button id="connectBtn">MetaMask 연결</button>
  <p>계정: <span id="account">-</span></p>
  <p>내 잔액: <span id="myBalance">-</span> / 게임 상금 준비금: <span id="pool">-</span></p>
```

전체 소스

web/odd-even.html (2/5) 28~54행

```
<button id="oddBtn" class="pick" disabled>홀</button>
<button id="evenBtn" class="pick" disabled>짝</button>
<p id="result"></p>
<p id="status"></p>

<h3>최근 게임</h3>
<ul id="history"></ul>

<p>배포한 계정만: <button id="withdrawBtn" disabled>남은 준비금 회수</button></p>

<script type="module">
  import { ethers } from "/node_modules/ethers/dist/ethers.min.js";

  const abi = [
    "function play(bool pickOdd) payable",
    "function BET() view returns (uint256)",
    "function withdraw()",
    "event Played(address indexed player, bool pickOdd, uint256 number, bool win)"
  ];

  let provider, signer, game;
  const $ = (id) => document.getElementById(id);
  const eth = (wei) => Number(ethers.formatEther(wei)).toFixed(6) + " ETH";   // 소수점 6자리까지

  // MetaMask 에서 네트워크를 바꾸면 새로 고침, 연결된 뒤 계정을 바꿔도 새로 고침 (페이지를 열 때 바로 등록)
  // (처음 연결을 승인할 때도 accountsChanged 가 오므로, 연결이 끝난 뒤에만 새로 고침)
  window.ethereum?.on("chainChanged", () => location.reload());
```

전체 소스

web/odd-even.html (3/5) 55~81행

```
window.ethereum?.on("accountsChanged", () => { if (game) location.reload(); });

$("connectBtn").onclick = async () => {
  try {
    provider = new ethers.BrowserProvider(window.ethereum);
    await provider.send("eth_requestAccounts", []);
    signer = await provider.getSigner();
    const { chainId } = await provider.getNetwork();
    const res = await fetch(`/ignition/deployments/chain-${chainId}/deployed_addresses.json`);
    if (!res.ok) {
      // 이 네트워크에는 배포 기록이 없음 → MetaMask 에 Sepolia(0xaa36a7 = 11155111)로 바꿔 달라고 요청
      $("status").textContent = `chain ${chainId} 에 배포 기록이 없습니다 → MetaMask에서 Sepolia로 전환해 주세요`;
      await provider.send("wallet_switchEthereumChain", [{ chainId: "0xaa36a7" }]);
      return;                       // 전환되면 chainChanged → 새로 고침 → 연결 버튼 다시 누르기
    }
    const address = (await res.json())["OddEvenModule#OddEven"];
    if (!address) throw new Error("OddEven 이 배포되지 않았습니다");
    game = new ethers.Contract(address, abi, signer);

    $("account").textContent = signer.address;
    $("connectBtn").disabled = true;
    $("oddBtn").disabled = $("evenBtn").disabled = false;
    $("withdrawBtn").disabled = false;
    $("status").textContent = "";   // 이전 오류 문구 지우기
    await refresh();
  } catch (e) {
    $("status").textContent = e.shortMessage ?? e.message;
```

전체 소스

web/odd-even.html (4/5) 82~108행

```
  }
};

$("oddBtn").onclick = () => play(true);
$("evenBtn").onclick = () => play(false);

async function play(pickOdd) {
  try {
    $("result").textContent = "";
    $("status").textContent = "MetaMask에서 확인해 주세요";
    // payable 함수: { value } 로 ETH 를 함께 보냄
    const value = await game.BET();
    // 가스는 보내기 전에 미리 실행해서 정함 → 실패할 거래(금액 틀림, 상금 부족)는 이유와 함께 멈춤, 보내지 않음
    const tx = await game.play(pickOdd, { value });
    $("status").textContent = "블록에 담기기를 기다리는 중…";
    const receipt = await tx.wait();

    // receipt.logs 에서 Played 이벤트 찾기 → 뽑힌 숫자, 승패
    const ev = receipt.logs.map((log) => game.interface.parseLog(log)).find((p) => p?.name === "Played");
    const { number, win } = ev.args;
    $("result").textContent = `뽑힌 숫자: ${number} (${number % 2n === 1n ? "홀" : "짝"})`;
    $("status").textContent = win ? "🎉 이겼습니다! 0.002 ETH를 받았습니다" : "❌ 졌습니다";
  } catch (e) {
    $("status").textContent = e.code === "ACTION_REJECTED" ? "MetaMask에서 거절함" : (e.reason ?? e.shortMessage ?? e.message);
  }
  await refresh();
}
```

전체 소스

web/odd-even.html (5/5) 109~137행

```
    // 남은 준비금 회수: 배포한 계정(owner)만 가능 → 다른 계정이면 미리 실행에서 "only owner" 로 멈춤, 보내지 않음
    $("withdrawBtn").onclick = async () => {
      try {
        $("status").textContent = "MetaMask에서 확인해 주세요";
        const tx = await game.withdraw();
        $("status").textContent = "블록에 담기기를 기다리는 중…";
        await tx.wait();
        $("status").textContent = "남은 준비금을 회수했습니다";
      } catch (e) {
        $("status").textContent = e.code === "ACTION_REJECTED" ? "MetaMask에서 거절함" : (e.reason ?? e.shortMessage ?? e.message);
      }
      await refresh();
    };

    async function refresh() {
      $("myBalance").textContent = eth(await provider.getBalance(signer.address));
      $("pool").textContent = eth(await provider.getBalance(await game.getAddress()));

      // 최근 5000 블록의 Played 이벤트 (가스비 X)
      const latest = await provider.getBlockNumber();
      const logs = await game.queryFilter(game.filters.Played(), Math.max(0, latest - 5000));
      $("history").innerHTML = logs.reverse().slice(0, 10).map((l) =>
        `<li>${l.args.player.slice(0, 6)}… ${l.args.pickOdd ? "홀" : "짝"} → ${l.args.number} ${l.args.win ? "🎉" : "✖"}</li>`
      ).join("");
    }
  </script>
</body>
</html>
```

전체 소스

scripts/serve.js 1~28행

```
// 웹 페이지를 브라우저에서 열기 위한 간단한 웹 서버
//   node scripts/serve.js   →   http://localhost:3000
// (MetaMask 는 file:// 로 연 페이지에서는 동작하지 않으므로 http 로 엽니다)
import http from "node:http";
import fs from "node:fs";
import path from "node:path";

const PORT = 3000;
const ROOT = process.cwd();   // 프로젝트 폴더 (hardhat-test)
const TYPES = { ".html": "text/html; charset=utf-8", ".js": "text/javascript", ".json": "application/json",
                ".css": "text/css", ".map": "application/json", ".svg": "image/svg+xml", ".png": "image/png" };
// 웹 페이지가 읽을 수 있는 폴더만 허용 (개인 키 등 다른 파일은 보내지 않음)
const ALLOWED = ["/web/", "/node_modules/ethers/dist/", "/ignition/deployments/", "/artifacts/contracts/"];

http.createServer((req, res) => {
  let url = decodeURIComponent(req.url.split("?")[0]);
  if (url === "/") url = "/web/index.html";
  const file = path.join(ROOT, url);
  if (!ALLOWED.some((p) => url.startsWith(p)) || !file.startsWith(ROOT) ||
      !fs.existsSync(file) || fs.statSync(file).isDirectory()) {
    res.writeHead(404); res.end("Not found"); return;
  }
  res.writeHead(200, { "Content-Type": TYPES[path.extname(file)] ?? "application/octet-stream",
                       "Cache-Control": "no-store" });
  fs.createReadStream(file).pipe(res);
}).listen(PORT, "127.0.0.1", () => {
  console.log(`웹 페이지: http://localhost:${PORT}   (끝내려면 Ctrl + C)`);
});
```

Q&A